EDUCONECTA · HANDOFF TÉCNICO

Guia de Arquitetura do Projeto

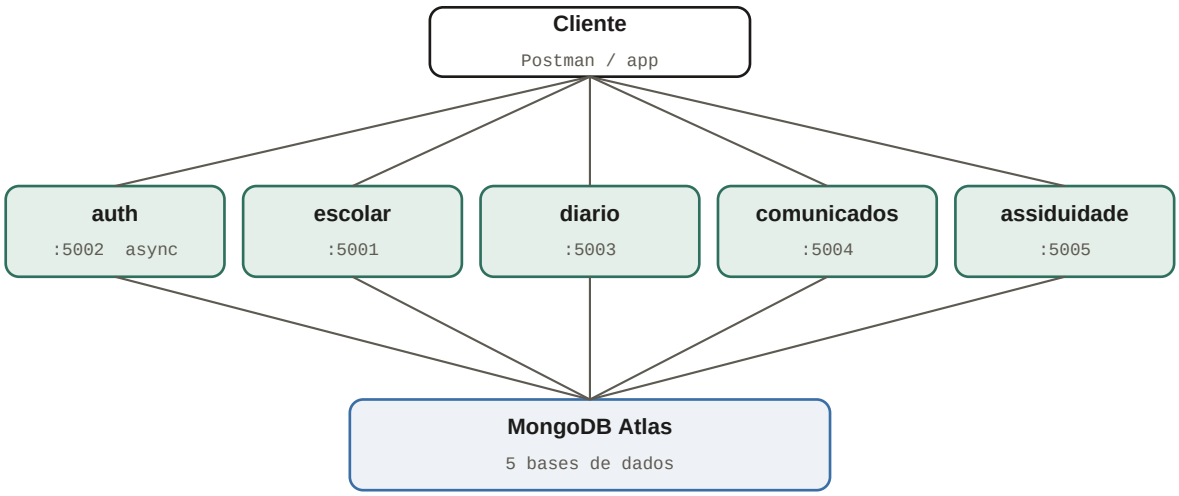
Documento de passagem de testemunho: o essencial para quem pegar no projeto e continuar o trabalho sem ter de redescobrir as decisões já tomadas.

EduConecta é uma aplicação de gestão escolar — comunicação entre escola, encarregados de educação e profissionais (docentes, educadoras, auxiliares). É construída como **5 microsserviços** independentes, cada um em Python/Flask (o **auth** é assíncrono, em Quart), organizados em **MVC**, todos ligados ao mesmo cluster **MongoDB Atlas** na cloud.

Estado atual: os 5 serviços correm em Docker Compose, todos ligados à Atlas, com 55 testes automatizados (pytest) a passar. O fluxo completo login → criar dados → persistir na Atlas foi testado manualmente via Postman.

1. Arquitetura geral

O cliente fala diretamente com cada serviço (não há gateway/API única). Dentro da rede Docker, os serviços de negócio consultam o **auth** para validar a sessão de quem fez o pedido, e o **comunicados**/**assiduidade** consultam também o **escolar** para confirmar relações (ex: este encarregado é mesmo encarregado deste aluno?).



Linhas sólidas: pedidos do cliente. Linhas finas: comunicação interna entre serviços (verificação de sessão/permissões) e ligação de cada serviço à sua base de dados na Atlas.

Serviço	Porta	Trata de
`auth`	5002	Contas, login, JWT, perfis, criação de contas pela administração, envio de credenciais
`escolar`	5001	Alunos, turmas, encarregados de educação, profissionais e associações
`diario`	5003	Registos diários de acompanhamento de cada aluno
`comunicados`	5004	Comunicados, conversas/chat, fotos e vídeos anexados
`assiduidade`	5005	Registo de presenças e faltas por turma

2. Autenticação e permissões

Login é feito só no **auth** (`POST /login`), que devolve um **JWT** válido por 15 minutos. Esse token vai no cabeçalho `Authorization: Bearer <token>` em todos os pedidos seguintes, a qualquer um dos 5 serviços.

Não há registo público — só uma conta **admin** pode criar outras contas (`POST /admin/utilizadores`), que recebem uma password temporária por email e são obrigadas a trocá-la no primeiro acesso.

Fluxo típico (exemplo: criar uma turma)

- Cliente faz `POST /login` ao **auth** → recebe o token.
- Cliente faz `POST /turmas` ao **escolar** com o token.
- **escolar** chama `GET /me` no **auth** (mesmo token) para confirmar quem é e que perfil tem.
- **escolar** confirma que o perfil tem permissão (ex: admin) e só depois grava na Atlas.

Perfis existentes

`admin`, `encarregado`, `docente`, `educadora`, `auxiliar`. Cada rota decide quais perfis pode aceitar (normalmente: admin sempre pode; outros perfis só nas rotas pessoais, ex: `/me/alunos` para encarregados).

3. Estrutura de código (MVC)

A mesma estrutura repete-se nos 5 serviços — só muda o conteúdo de `models/` e `controllers/`.

```
<servico>/
├── app.py cria a app, regista rotas
├── db.py liga ao MongoDB Atlas
├── models/ acesso a dados
├── controllers/ rotas + validacao
├── utils/ tokens, chamadas a outros servicos
└── tests/ pytest (BD mockada)
```

Regra de ouro ao mexer no código

- Lógica de acesso a dados (queries MongoDB) vive em `models/`, nunca direto no controller.
- Validação de input, permissões e formato da resposta vivem em `controllers/`.
- Chamadas a outros serviços (urllib) e utilitários partilhados vivem em `utils/`.
- `app.py` só cria a app e regista as rotas — nunca deve crescer com lógica de negócio.

Nota: o **auth** foi o piloto de uma migração para assíncrono (Quart + PyMongo `AsyncMongoClient` + `aiosmtplib`). Os outros 4 serviços ficaram em Flask síncrono por decisão de âmbito — dá para converter depois, seguindo o mesmo padrão do **auth** como referência.

4. Base de dados (MongoDB Atlas)

Todos os serviços ligam ao **mesmo cluster Atlas**, cada um com a sua própria base de dados lá dentro — não há isolamento de utilizador por serviço (é usado um único utilizador admin do Atlas, decisão deliberada para simplificar um projeto académico). As bases de dados e coleções são criadas **automaticamente** na primeira escrita — não é preciso nenhum passo de "criar tabela".

Bases de dados

- `auth_db` — coleção `utilizadores`
- `escolar_dc` — `alunos`, `turmas`, `encarregados_educacao`, `profissionais`
- `diario_db`, `comunicados_db`, `educonecta_assiduidade` — uma por serviço

Onde estão as credenciais

Não estão neste documento, de propósito. Estão em dois ficheiros na raiz do projeto (`EduConecta_projeto/`), ambos fora do git (`.gitignore`): `.env` (variáveis usadas pelo Docker Compose) e `atlas-credentials.env` (exportado pelo onboarding da Atlas). Pede-os diretamente a quem já tem o projeto a correr — nunca os reenvies por email/chat sem necessidade.

5. Como pôr o projeto a correr

1. Variáveis de ambiente

Criar `.env` na raiz de `EduConecta_projeto/` com as variáveis abaixo, e `.env.smtp` com `SMTP_USERNAME`/`SMTP_PASSWORD`.

Variável	Para quê
`MONGO_URI`	Connection string da Atlas (partilhada pelos 5 serviços)
`MONGO_AUTH_DATABASE`	Nome da BD do auth (auth_db)
`MONGO_ESCOLAR_DATABASE`	Nome da BD do escolar (escolar_dc)
`MONGO_DIARIO_DATABASE`	Nome da BD do diario (diario_db)
`MONGO_COMUNICADOS_DATABASE`	Nome da BD do comunicados (comunicados_db)
`MONGO_ASSIDUIDADE_DATABASE`	Nome da BD do assiduidade (educonecta_assiduidade)
`JWT_SECRET_KEY`	Chave de assinatura dos tokens (auth), mínimo 32 caracteres
`SMTP_USERNAME / SMTP_PASSWORD`	Em .env.smtp — envio de credenciais por email (auth)

2. Arrancar tudo

A partir de `EduConecta_projeto/`:

```
docker compose up --build -d
docker compose ps
curl http://127.0.0.1:5002/health
```

3. Portas

escolar :5001 · auth :5002 · diario :5003 · comunicados :5004 · assiduidade :5005. Todos têm `/health` e `/ready` (este último confirma a ligação à Atlas).

4. Testar com Postman

- `POST /login` no auth (:5002) com email + password → guarda o `access_token`.
- Usa esse token como Bearer Token nos pedidos aos outros serviços.
- Dica: num script de "Tests" do pedido de login, `pm.environment.set("token", pm.response.json().access_token)` e usa `{{token}}` nos outros pedidos.

5. Correr os testes automáticos

Por serviço, a partir de `backend/services/<servico>/`:

```
pip install -r requirements-dev.txt
pytest -q
```

6. O que falta / pontos em aberto

Lista honesta do que não está feito, para quem continuar não perder tempo a descobrir:

- **Async nos outros 4 serviços** — só o auth foi convertido para Quart/async; escolar, diario, comunicados e assiduidade continuam em Flask síncrono.
- **Sem endpoint para listar profissionais de uma turma** — existe associação (`PUT /turmas/<id>/profissionais/<id>`) mas não há `GET /turmas/<id>/profissionais` nem `GET /turmas/<id>` para consultar isso depois.
- **Scripts `teste_*.py` na raiz do projeto** (fora dos serviços) foram escritos antes da reestruturação MVC e da migração para a Atlas — provavelmente já não funcionam sem ajustes; os testes atuais e válidos são os `pytest` dentro de cada `backend/services/<servico>/tests/`.
- **Ficheiros `.bak`** antigos (ex: `app.py.antes_associacoes.bak`) ainda existem em alguns serviços — são versões anteriores do app.py, podem ser apagados com segurança se já não forem precisos.
- **Um único utilizador Atlas para tudo** — simplificação deliberada; separar por serviço é possível mais tarde se for preciso reforçar isolamento.
- **Sem CI** — os testes correm manualmente; não há pipeline a correr `pytest` automaticamente a cada alteração.

Resumo do estado

5 microsserviços em Docker, todos `healthy`, todos ligados à Atlas, 55 testes automáticos a passar, fluxo de autenticação e criação de dados validado manualmente ponta a ponta.